



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería del Software

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

MÁLAGA, March 11, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADUADO EN INGENIERÍA DE LA SALUD

Mención en Bioinformática

Biblioteca extensible de optimización metaheurística en Rust

**Extensible metaheuristic optimization library in
Rust**

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, MARCH 11, 2026

Fecha defensa: Septiembre de 2025

Resumen

El presente Trabajo de Fin de Grado tiene como objetivo principal el diseño y desarrollo de una biblioteca de optimización metaheurística de carácter extensible y modular, basada en una filosofía de cero dependencias y programada íntegramente en Rust, un lenguaje de programación compilado orientado a la seguridad de memoria y al alto rendimiento.

Palabras clave: Rust, Metaheurísticas, Optimización, Biblioteca

Abstract

Keywords: Rust, Metaheuristics, Optimization, Library

Agradecimientos

(quiero poner un agradecimiento, esto lo haré lo último

Resumen	1
Abstract	3
Agradecimientos	5
1 Introducción	13
1.1 Motivación	13
1.2 Objetivos	14
1.3 Tecnologías a utilizar	14
2 Metodología de trabajo	17
2.1 Modelo de desarrollo	17
2.2 Ecosistema y herramientas de desarrollo	17
2.3 Fases del proyecto	17
2.3.1 Investigación y capacitación	17
2.3.2 Definición de requisitos y planificación	18
2.3.3 Implementación técnica	18
3 Optimización y Metaheurísticas	21
4 Desarrollo	23
4.1 Arquitectura	23
4.2 Módulos	23
4.2.1 Algoritmos	24
4.2.2 Observadores	24
4.2.3 Operadores	25
4.2.4 Problemas	26
4.2.5 Indicadores de calidad	26
4.2.6 Conjuntos de soluciones	26
4.2.7 Soluciones	26
4.2.8 Utilidades	27
5 Validación experimental	29
6 Conclusiones y líneas futuras	31
Apéndice A. Installation Manual	33

A.1	Requisitos	33
A.2	Instalación desde el repositorio	33
A.3	Instalación como <i>crate</i> mediante Cargo	34

Índice de Figuras

1.1	Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [8].	15
4.1	Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [10].	23

Nomenclatura

framework conjunto estandarizado de herramientas, librerías, reglas y buenas prácticas prediseñadas para agilizar el desarrollo de software

SCRUM marco de trabajo ágil para gestionar proyectos complejos

1

Introducción

1.1 Motivación

La optimización metaheurística constituye una herramienta fundamental para la resolución de problemas complejos en aquellos ámbitos donde los métodos exactos resultan inviables, ya sea debido al elevado coste computacional o a la naturaleza no lineal y no convexa de los espacios de búsqueda. Este tipo de técnicas ha demostrado su utilidad en una amplia variedad de aplicaciones, como la planificación y asignación de recursos, la optimización de rutas, el diseño de sistemas, la inteligencia artificial, el aprendizaje automático y la ingeniería en general.

En distintos lenguajes de programación existen bibliotecas maduras que facilitan la aplicación de técnicas metaheurísticas. Por ejemplo, *jMetal* es un framework ampliamente reconocido para la optimización multiobjetivo con metaheurísticas [6]. Su versión en Python, *jMetalPy*, ofrece un entorno extensible y orientado a objetos para experimentar con técnicas clásicas y avanzadas de optimización [4]. Asimismo, *MEALPY* (MEta-heuristic ALgorithms in PYthon) proporciona implementaciones de numerosos algoritmos inspirados en la naturaleza, tanto clásicos como híbridos, para abordar problemas complejos [11]. Sin embargo, dentro del ecosistema de Rust, el soporte para la optimización metaheurística es todavía limitado, lo que dificulta la adopción de estas metodologías en proyectos desarrollados en dicho lenguaje y pone de manifiesto la necesidad de disponer de herramientas nativas que se integren de forma natural con sus principios de diseño, especialmente en lo relativo al rendimiento, la seguridad de memoria y el control de recursos.

Este Trabajo de Fin de Grado surge con la motivación de dar respuesta a esta necesidad mediante el desarrollo de una biblioteca de optimización metaheurística modular y extensible. No obstante, abordar un proyecto de esta envergadura resulta especialmente desafiante en las etapas iniciales, particularmente en ausencia de experiencia previa. Afrontar un trabajo de fin de carrera implica

recopilar y consolidar los conocimientos adquiridos a lo largo de la titulación y aplicarlos en un proyecto real, lo cual supone una tarea de notable complejidad. Por ello, la elección de un tema que despierte interés, curiosidad y motivación personal resulta un factor clave para el desarrollo del proyecto.

El diseño y desarrollo de algoritmos metaheurísticos conforma un campo especialmente atractivo, al combinar fundamentos de la teoría de la computación, la optimización y la inteligencia artificial. Su aplicación en dominios tan diversos como la logística, la ingeniería o la biología computacional pone de manifiesto su versatilidad y su impacto en la resolución de problemas reales.

En el momento de inicio de este proyecto, no existían bibliotecas consolidadas orientadas al desarrollo de este tipo de soluciones en el lenguaje Rust, lo que constituyó una motivación adicional para su realización. Asimismo, el propio lenguaje Rust actuó como detonante del proyecto, al tratarse de una tecnología moderna, diseñada con un fuerte énfasis en la seguridad de memoria y el rendimiento, y especialmente adecuada tanto para el aprendizaje como para el desarrollo de software robusto y eficiente.

1.2 Objetivos

El objetivo principal de este proyecto es desarrollar una biblioteca en Rust que implemente algoritmos metaheurísticos para la optimización de problemas complejos. Debe ser capaz de resolver correctamente multitud de problemas de único objetivo y un conjunto pequeño de problemas multiobjetivo. Idealmente, esta biblioteca debería ser fácil de usar, eficiente y extensible, permitiendo a los usuarios adaptar los algoritmos a sus necesidades específicas. Para garantizar el cumplimiento de estas premisas, se ha realizado un desglose exhaustivo de los requisitos del sistema, donde se definen formalmente las funcionalidades y restricciones que guiarán el desarrollo de la herramienta.

1.3 Tecnologías a utilizar

Como lenguaje de programación, se ha elegido **Rust**, debido a su enfoque en el rendimiento, por su capacidad de manejo eficiente de la concurrencia y diferentes hilos, lo que lo hace ideal para la implementación de algoritmos heurísticos complejos que a menudo requieren un procesamiento paralelo intensivo. Además, su creciente ecosistema y su tipo de sistema robusto nos permiten construir soluciones de optimización escalables y de alto rendimiento. También, Rust ha comenzado a consolidarse como uno de los lenguajes de programación utilizados en el desarrollo del *kernel* de Linux, dejando atrás su carácter experimental y siendo aprobado oficialmente para la implementación de determinados componentes del núcleo, lo que refuerza su madurez, fiabilidad y adecuación para sistemas críticos [7].

Uno de los principales motivos por el que Rust es tan buena elección es por su gestor de paquetes y compilación, **Cargo**, que facilita la gestión de dependencias y la construcción de proyectos, permitiendo a los desarrolladores centrarse en la implementación de algoritmos en lugar de en la configuración del entorno.

Este gestor de paquetes se encuentra entre los más populares y admirados por los desarrolladores, como se puede observar en la estadística de la encuesta anual de Stack Overflow, donde Cargo destaca como uno de los gestores de paquetes más valorados para el desarrollo e infraestructura en la nube durante 2025 [8].

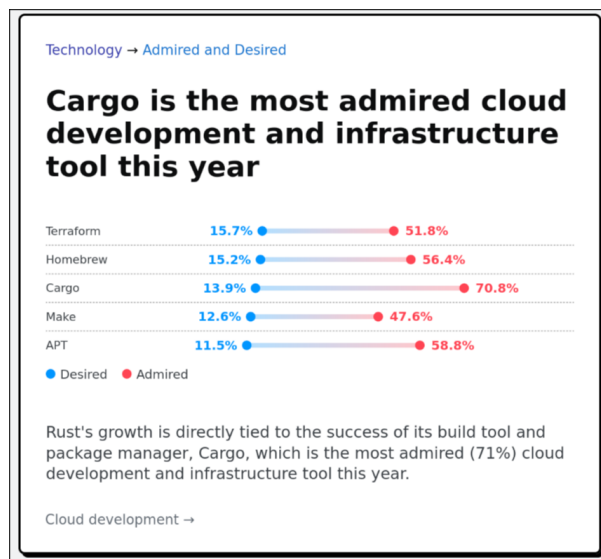


Figura 1.1. Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [8].

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover**, de **JetBrains**, que es un IDE específico para Rust. Esto fue debido a que en ese momento los conocimientos sobre Rust eran muy limitados y se necesitaba un entorno más guiado e invasivo para poder progresar.

Para la documentación y redacción del proyecto, se ha utilizado $\text{\LaTeX}$, para el control de versiones **Git** junto con **GitHub** como plataforma de alojamiento remoto.

2

Metodología de trabajo

En esta sección se describen los procedimientos, herramientas y fases seguidas para la consecución de los objetivos del proyecto.

2.1 Modelo de desarrollo

Para el desarrollo del proyecto se ha adoptado una metodología iterativa, inspirada en el marco de trabajo SCRUM, pero adaptada a las particularidades de un entorno de trabajo unipersonal. Esta adaptación permite conservar las ventajas de la planificación incremental y la revisión continua, ajustándolas a la naturaleza y alcance del proyecto.

Como herramienta de apoyo a la gestión y seguimiento de las tareas se ha utilizado JIRA, cuyo uso y configuración dentro del proyecto se detallarán con mayor profundidad en el apartado siguiente.

2.2 Ecosistema y herramientas de desarrollo

La selección de las herramientas de trabajo se realizó buscando maximizar la eficiencia en la detección de errores y la navegación por el código. Se utilizó mayoritariamente Visual Studio Code como entorno de desarrollo principal, aprovechando su versatilidad y el robusto soporte proporcionado por la extensión *rust-analyzer* [2].

2.3 Fases del proyecto

Un proyecto como el que nos encontramos debe estructurarse en varias partes, en este caso en tres etapas diferenciadas:

2.3.1 Investigación y capacitación

Debido a la pronunciada curva de aprendizaje de Rust, esta fase inicial se centró en la asimilación de sus fundamentos sintácticos y las particularidades de su compilador. Para consolidar estos conceptos,

se llevó a cabo la migración de un proyecto personal previo (una aplicación de escritorio), trasladando su arquitectura basada en Java y Javalin hacia un ecosistema nativo en Rust mediante el framework Tauri [9].

Paralelamente, se desarrolló un repositorio de pruebas a modo de entorno de experimentación, destinado a la implementación de pequeños módulos funcionales que permitieron dominar aún más el lenguaje. Esta etapa de formación técnica se complementó con un estudio exhaustivo sobre metaheurísticas y algoritmos genéticos, sentando las bases teóricas necesarias para el posterior diseño de la biblioteca.

Dentro del estudio técnico, cobró especial relevancia el análisis del repositorio y de la arquitectura de jMetal [6], un framework de optimización metaheurística multiobjetivo escrita en java por investigadores de la universidad de Málaga, ayudó a comprender los patrones de diseño y las estructuras de datos necesarias para implementar metaheurísticas de forma extensible.

2.3.2 Definición de requisitos y planificación

Durante esta etapa se realizó un trabajo de introspección, se analizaron los objetivos y se elaboró un documento donde se especificaron todos los requisitos del sistema.

A partir de estos requisitos, se creó un *backlog* en Jira, donde se registraron todas las tareas necesarias para completar el proyecto. Posteriormente, las tareas se organizaron en seis “sprints” y se clasificaron en las columnas “Por hacer”, “En curso” y “Por revisar”. Esta organización permitió un seguimiento claro del progreso y facilitó la comunicación con el tutor, mostrando periódicamente las tareas en la columna “Por revisar” para recibir su aprobación. Una vez que el tutor daba el visto bueno, las tareas se marcaban como “Listo”, asegurando un flujo de trabajo controlado y transparente [1].

2.3.3 Implementación técnica

Una vez definidos los requisitos del sistema y establecida la planificación del proyecto, se procedió a la implementación técnica de la biblioteca de optimización metaheurística. Esta fase tuvo como objetivo materializar las decisiones de diseño adoptadas previamente, dando lugar a un sistema funcional, modular y extensible, desarrollado íntegramente en el lenguaje Rust.

El proceso de implementación se llevó a cabo de forma incremental, permitiendo validar progresivamente las abstracciones definidas y adaptar la arquitectura a las particularidades del lenguaje y a los requisitos detectados durante el desarrollo.

La implementación estuvo fuertemente condicionada por las características propias del lenguaje Rust. La ausencia de herencia clásica y el uso intensivo de *traits* influyeron directamente en el diseño

de las abstracciones, fomentando un enfoque basado en la composición y el polimorfismo estático.

Asimismo, el sistema de propiedad y préstamos de Rust actuó como una herramienta de validación temprana, obligando a definir de forma explícita la gestión de memoria y el ciclo de vida de las estructuras de datos. Aunque este modelo supuso una dificultad inicial, permitió garantizar un alto nivel de seguridad y robustez en la implementación final.

La implementación de la biblioteca se realizó siguiendo un orden lógico que permitió construir el sistema de forma progresiva. En primer lugar, se desarrollaron las abstracciones básicas, tales como las representaciones de problemas y soluciones. Posteriormente, se implementaron los algoritmos de optimización, seguidos de los operadores fundamentales y de los mecanismos de observación.

Este enfoque permitió validar cada componente de manera aislada antes de integrarlo en el sistema global, reduciendo la complejidad del proceso de depuración y facilitando la detección de errores conceptuales en fases tempranas del desarrollo.

Durante la fase de implementación se llevó a cabo una validación continua del sistema, apoyándose principalmente en el compilador de Rust como herramienta de detección de errores y en la ejecución de pruebas y ejemplos controlados para verificar el comportamiento de los algoritmos.

Asimismo, se realizaron refactorizaciones periódicas del código con el objetivo de mejorar su claridad, reducir duplicidades y adaptar la arquitectura a nuevas necesidades detectadas durante el desarrollo. Este proceso iterativo permitió alcanzar una implementación estable y coherente con los objetivos iniciales del proyecto.

3

Optimización y Metaheurísticas

4

Desarrollo

4.1 Arquitectura

El diseño y modelado de la arquitectura representan la fase de mayor complejidad y dedicación en el desarrollo. La integridad de esta estructura determina la calidad técnica, eficiencia y mantenibilidad de la biblioteca. Con el objetivo de garantizar un nivel de abstracción óptimo que permita su aplicación en diversos casos de uso, se han integrado los siguientes criterios de diseño:

Los límites del problema no se encuentran en el algoritmo sino en el problema

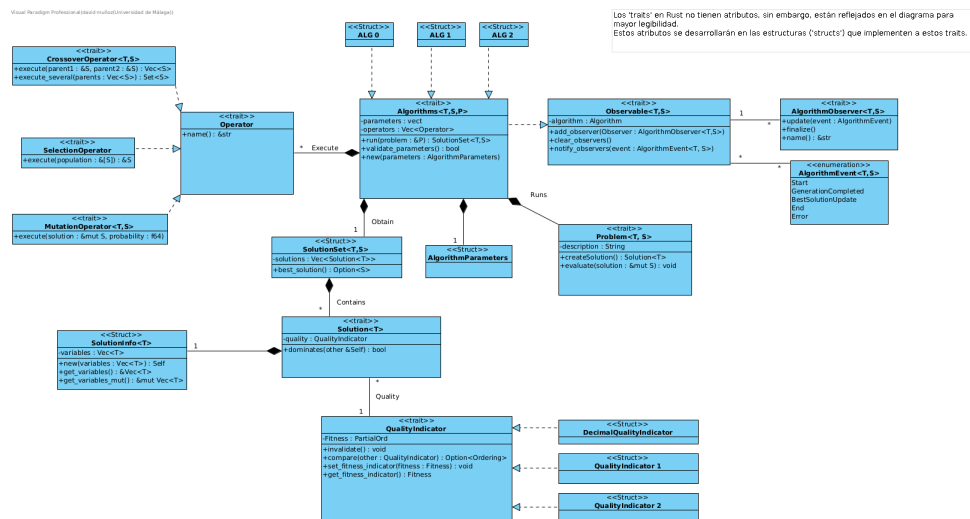


Figura 4.1. Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [10].

4.2 Módulos

A continuación, se describen los distintos módulos que componen la biblioteca y las responsabilidades asociadas a cada uno de ellos.

4.2.1 Algoritmos

Este módulo es el encargado de ejecutar los distintos problemas definidos en la biblioteca mediante la función `run`, la cual recibe como argumento una referencia al problema que se desea resolver. Para llevar a cabo su funcionamiento, los algoritmos requieren la aplicación de *operadores*, responsables de realizar las transformaciones necesarias sobre las soluciones con el objetivo de explorarlas y mejorarlas de forma iterativa.

Gracias a la arquitectura modular adoptada, dichos operadores son intercambiables, lo que permite adaptar el comportamiento del algoritmo sin alterar su implementación interna. De igual modo, esta modularización facilita la sustitución o modificación del problema sobre el que se ejecuta el algoritmo de manera sencilla y flexible.

Asimismo, los algoritmos admiten la incorporación directa de *observadores*, cuyo propósito es analizar y monitorizar su comportamiento durante la ejecución. Estos observadores se describen con mayor detalle en su apartado correspondiente.

Estos algoritmos pueden ejecutarse de forma concurrente o paralela, en función de la configuración seleccionada por el usuario. En este trabajo se ofrece la posibilidad de especificar manualmente el número de hilos a crear o, alternativamente, delegar esta decisión al propio programa. En este último caso, el número de hilos se determina automáticamente en función de las características del sistema en el que se ejecuta, haciendo uso de la función `available_parallelism()` proporcionada por la librería estándar.

4.2.2 Observadores

Los observadores estarán asociados a aquellos algoritmos que implementen el *trait Observable*. Dicho *trait* proporciona la funcionalidad necesaria para añadir y eliminar observadores.

Para la comunicación entre el algoritmo y los observadores adjuntados, se ha diseñado un sistema basado en canales (*channels*), un patrón idiomático en Rust para la comunicación segura entre hilos. Inicialmente, se planteó una implementación en la que el *trait Observable* incluyese un método encargado de notificar directamente a todos los observadores registrados. Sin embargo, debido a las restricciones impuestas por el sistema de propiedad y préstamos de Rust (*Borrow Checker*), esta aproximación resultó compleja de implementar de forma clara y segura.

Como alternativa, se optó por un modelo concurrente en el que el algoritmo de optimización se ejecuta en un hilo independiente, mientras que los observadores se mantienen a la espera en hilos separados, recibiendo de manera asíncrona la información enviada por el algoritmo a través de los

canales. Este enfoque simplifica la gestión de la concurrencia, evita conflictos de préstamos y garantiza una comunicación segura entre los distintos componentes del sistema.

————— FALTA

Existen distintos tipos de observadores, entre los que se incluyen aquellos que muestran la información directamente por terminal o los que generan tablas y estructuras de datos más elaboradas. Estos observadores permiten analizar la evolución del proceso de optimización en función del número de evaluaciones realizadas, proporcionando al usuario una visión clara y detallada del comportamiento del algoritmo.

————— IDEAS

Los ChartObservers, generarán varios gráficos, uno que muestre como avanza la calidad de las soluciones junto a las evaluaciones, como avanzan las evaluaciones y uno con las soluciones en general. En el caso de que las soluciones sean un vector bidimensional, un gráfico de puntos, donde estos estén en las coordenadas de los objetivos, en el caso de que no sea un vector, pues un gráfico de barras, con las veces que se ha repetido cada valor y si fuera una vector de más de 2 dimensiones, podríamos generar varios gráficos con los diferentes casos. Se plantea generar un html con toda la información super currado.

4.2.3 Operadores

Este componente constituye el núcleo de la biblioteca de optimización, ya que encapsula la lógica fundamental de los algoritmos evolutivos implementados. En particular, se definen tres tipos principales de operadores, cada uno de los cuales desempeña un papel esencial dentro del proceso de optimización:

- **Operadores de mutación:** introducen variaciones aleatorias en los individuos de la población con el objetivo de mantener la diversidad genética y evitar la convergencia prematura hacia soluciones subóptimas. Estos operadores permiten explorar nuevas regiones del espacio de búsqueda mediante modificaciones controladas de las soluciones existentes.
- **Operadores de selección:** se encargan de elegir los individuos que participarán en la generación de nuevas soluciones, generalmente en función de su calidad o valor de aptitud. Su función principal es favorecer la propagación de las mejores soluciones encontradas hasta el momento, equilibrando la presión selectiva con la preservación de la diversidad poblacional.
- **Operadores de cruce:** combinan la información de dos o más individuos seleccionados para

generar nuevos descendientes. A través de este proceso se busca explotar las características favorables de las soluciones existentes, recombiniéndolas de manera que puedan dar lugar a individuos con un mejor desempeño.

4.2.4 Problemas

Este módulo contiene la información del problema que se quiere resolver. El “trait” que deben implementar todos los problemas es muy sencillo, solo obliga a crear unas simples funciones para obtener y editar la descripción del problema en sí y otras funciones para evaluar las propias soluciones al problema y otra para crear una solución de la que partirá el algoritmo que intente solventar el problema.

Los problemas pueden ser muy diversos, por tanto no es fácilmente delimitable este módulo. Para ayudar a diseñar problemas en esta biblioteca, se muestran ejemplos de estos problemas e implementaciones para ayudar a la construcción de estos con el patrón “Builder”.

4.2.5 Indicadores de calidad

Es una abstracción superior para administrar la calidad de las soluciones para casos más específicos o multiobjetivo.

4.2.6 Conjuntos de soluciones

Los algoritmos devolverán esta estructura de datos, es un “wrapper” que contiene cada mejor solución obtenida por cada hebra por el algoritmo al intentar solucionar el problema.

4.2.7 Soluciones

Las soluciones representan las estructuras que almacenan el resultado generado por la aplicación iterativa de los operadores dentro de los algoritmos de optimización.

En una primera aproximación del diseño, `Solution` se definió como un *trait* que declaraba un conjunto de funciones fundamentales, tales como la obtención del valor de *fitness*, el acceso a las variables de la solución o la recuperación de su representación interna. Este *trait* debía ser implementado por distintos *structs*, cada uno adaptado a las necesidades específicas de cada problema.

Por ejemplo, si el problema requería representar una solución como una permutación binaria, el *struct* correspondiente contendría como atributos un vector binario junto con el valor de calidad asociado a dicha solución. Sin embargo, se observó que esta arquitectura introducía una elevada fragmentación en las implementaciones y dificultaba la generalización de la biblioteca para soportar un mayor número de problemas y algoritmos.

Por este motivo, se optó por una solución más abstracta y flexible. En el diseño actual, `Solution` se define como un *struct* genérico parametrizado por dos tipos: `T` y `Q`. El tipo `T` representa el tipo de dato de las variables de decisión almacenadas en la solución, mientras que `Q` representa el tipo asociado a la evaluación de calidad de dicha solución.

Para poder comparar soluciones entre sí, el tipo `Q` debe implementar un *trait* personalizado denominado `Dominance`. Este *trait* define el comportamiento necesario para establecer relaciones de dominancia entre valores de calidad, proporcionando un método que permite comparar dos instancias del mismo tipo y determinar cuál de ellas representa una solución superior.

Adicionalmente, la estructura `Solution` expone un método denominado `dominates`, cuya implementación delega la comparación en el método definido por el *trait* `Dominance`. De esta manera, la lógica de comparación queda desacoplada de la representación de la solución, permitiendo soportar tanto optimización monoobjetivo como multiobjetivo mediante diferentes implementaciones del tipo `Q`.

4.2.8 Utilidades

Para posibilitar el desarrollo de este proyecto y cumplir el objetivo de mantener una arquitectura con *cero dependencias externas*, se han implementado pequeños módulos auxiliares utilizados en el núcleo de la biblioteca. Estos módulos reproducen de forma ligera y controlada parte de la funcionalidad ofrecida por `crates` ampliamente utilizados en el ecosistema Rust, como `rand` [3] o `plotters` [5]. De este modo, se evita la inclusión de bibliotecas externas sin renunciar a las capacidades necesarias para el correcto funcionamiento del sistema.

5

Validación experimental

Experimentos con varios problemas

6

Conclusiones y líneas futuras

El desarrollo de este software ha representado un cambio de paradigma respecto a mi formación académica previa. La complejidad técnica no radica meramente en la sintaxis, sino en la comprensión profunda del ecosistema del lenguaje, así como de sus virtudes y limitaciones intrínsecas.

Rust supuso un reto, un lenguaje de sistemas que, si bien permite ciertos patrones de diseño, se aleja de la Programación Orientada a Objetos convencional de lenguajes como Java a la cual estaba acostumbrado. En su lugar, propone un modelo basado en estructuras (structs) y rasgos (traits), estos últimos con una clara influencia de las clases de tipos de Haskell.

Estas variaciones desembocaron en problemas durante el desarrollo de la arquitectura, al no existir tampoco herencia, un factor limitante en el desarrollo de la arquitectura

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [2]. Durante el desarrollo más temprano, también se usó **Rust Rover**, de **JetBrains**,

Apéndice A. Installation

Manual

En esta sección se describe el proceso de instalación y puesta en marcha de la biblioteca de optimización metaheurística desarrollada en este trabajo. Dado que el proyecto ha sido implementado íntegramente en Rust, su integración en otros proyectos resulta sencilla y se ajusta a los mecanismos estándar proporcionados por el propio ecosistema del lenguaje.

A.1 Requisitos

Para poder utilizar la biblioteca es necesario disponer de los siguientes elementos:

- Un sistema operativo compatible con el compilador de Rust (Linux, macOS o Windows).
- El compilador de Rust y su gestor de paquetes **Cargo**, instalados y correctamente configurados.
- Acceso a Internet para la descarga del código fuente o del *crate* correspondiente.

La instalación del entorno de desarrollo de Rust puede realizarse siguiendo las instrucciones oficiales proporcionadas por el lenguaje.

A.2 Instalación desde el repositorio

El método más directo para utilizar la biblioteca consiste en clonar el repositorio desde la plataforma GitHub y compilar el proyecto de forma local. Este enfoque resulta especialmente útil para desarrolladores que deseen inspeccionar, modificar o extender el código fuente.

El repositorio puede descargarse ejecutando el siguiente comando:

```
git clone https://github.com/DRLKs/rMetal
```

Una vez clonado el repositorio, basta con acceder al directorio del proyecto y ejecutar:

```
cargo build
```

Este comando descargará automáticamente las dependencias necesarias y generará la biblioteca en modo desarrollo. Para ejecutar ejemplos incluidos en el repositorio o realizar pruebas, puede utilizarse el comando:

```
cargo run
```

A.3 Instalación como *crate* mediante Cargo

Dado que la biblioteca ha sido diseñada siguiendo las convenciones estándar del ecosistema Rust, también puede integrarse fácilmente como dependencia en cualquier proyecto Rust mediante el gestor de paquetes **Cargo**. Este método es el más recomendado para usuarios que deseen emplear la biblioteca como componente dentro de una aplicación mayor.

Para ello, basta con añadir la dependencia correspondiente en el archivo **Cargo.toml** del proyecto:

```
[dependencies]
rMetal = "0.1.0"
```

Una vez añadida la dependencia, Cargo se encargará automáticamente de descargar, compilar e integrar la biblioteca durante el proceso de construcción del proyecto. A partir de ese momento, la funcionalidad proporcionada por la biblioteca podrá utilizarse directamente desde el código fuente mediante las instrucciones `use crate::` habituales del lenguaje.

Este mecanismo de distribución facilita la reutilización del software desarrollado y permite mantener actualizada la biblioteca de forma sencilla a medida que se publiquen nuevas versiones.

Bibliografía

- [1] Atlassian. *JIRA: Issue and Project Tracking Software*. Version 9.0. Herramienta para gestión de proyectos, seguimiento de incidencias y colaboración en equipo. URL: <https://www.atlassian.com/software/jira> (visited on 02/17/2026).
- [2] The rust-analyzer authors. *rust-analyzer*. URL: <https://github.com/rust-lang/rust-analyzer> (visited on 02/09/2026).
- [3] The Rand Project Developers and The Rust Project Developers. *rand: Rust random number generators and utilities*. Version 0.10.0. Crate for random number generation in Rust (MIT OR Apache-2.0). URL: <https://github.com/rust-random/rand> (visited on 02/17/2026).
- [4] J. J. Durillo and A. J. Nebro. *jMetalPy*. URL: <https://github.com/jMetal/jMetalPy> (visited on 02/09/2026).
- [5] Hao Hou, Aaron Erhardt, and contributors. *Plotters: A Rust drawing and plotting library*. Version 0.3.7. Librería Rust para generar gráficos y visualizaciones de datos. URL: <https://github.com/plotters-rs/plotters> (visited on 02/17/2026).
- [6] A. J. Nebro J. J. Durillo. *jMetal*. URL: <https://github.com/jMetal/jMetal> (visited on 02/09/2026).
- [7] Thorsten Leemhuis. *Linux Kernel: Rust Support Officially Approved*. heise online. Dec. 2025. URL: <https://www.heise.de/en/news/Linux-Kernel-Rust-Support-Officially-Approved-11109808.html> (visited on 02/15/2026).
- [8] Stack Overflow. *Stack Overflow Developer Survey 2025*. URL: <https://survey.stackoverflow.co/2025/> (visited on 10/12/2025).
- [9] David Muñoz del Valle. *PokerHelper*. URL: <https://github.com/DRLKs/PokerHelper> (visited on 02/09/2026).
- [10] Visual Paradigm International. *Visual Paradigm Professional*. Version 17.0. Herramienta profesional para modelado UML, diseño de arquitectura software y gestión del ciclo de vida del desarrollo. URL: <https://www.visual-paradigm.com> (visited on 02/17/2026).

- [11] Xin-She Yang and colleagues. *MEALPY: MEta-heuristic ALgorithms in PYthon*. URL: <https://mealpy.readthedocs.io/en/latest/pages/general/introduction.html> (visited on 02/09/2026).



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga